



Using Vision or Motion Capture Systems for Position

[Menu](#)[On this page](#)

Visual Inertial Odometry (VIO) and Motion Capture (MoCap) systems allow vehicles to navigate when a global position source is unavailable or unreliable (e.g. indoors, or when flying under a bridge. etc.).

Both VIO and MoCap determine a vehicle's *pose* (position and attitude) from "visual" information. The main difference between them is the frame perspective:

- VIO uses *onboard sensors* to get pose data from the vehicle's perspective (see [egomotion](#)).
- MoCap uses a system of *off-board cameras* to get vehicle pose data in a 3D space (i.e. it is an external system that tells the vehicle its pose).

Pose data from either type of system can be used to update a PX4-based autopilot's local position estimate (relative to the local origin) and also can optionally also be fused into the vehicle attitude estimation. Additionally, if the external pose system also provides linear velocity measurements, it can be used to improve the state estimate (fusion of linear velocity measurements is only supported by the EKF2).

This topic explains how to configure a PX4-based system to get data from MoCap/VIO systems (either via ROS or some other MAVLink system) and more specifically how to set up MoCap systems like VICON and Optitrack, and vision-based estimation systems like [ROVIO](#), [SVO](#) and [PTAM](#)).

INFO

The instructions differ depending on whether you are using the EKF2 or LPE estimator.

PX4 MAVLink Integration

PX4 uses the following MAVLink messages for getting external position information, and maps them to [uORB topics](#):

MAVLink	uORB
VISION_POSITION_ESTIMATE	vehicle_visual_odometry
ODOMETRY (frame_id = MAV_FRAME_LOCAL_FRD)	vehicle_visual_odometry
ATT_POS_MOCAP	vehicle_mocap_odometry
ODOMETRY (frame_id = MAV_FRAME_MOCAP_NED)	vehicle_mocap_odometry

EKF2 only subscribes to [vehicle_visual_odometry](#) topics and can hence only process the first two messages (a MoCap system must generate these messages to work with EKF2). The odometry message is the only message that can send also linear velocities to PX4. The LPE estimator subscribes to both topics, and can hence process all the above messages.

TIP

EKF2 is the default estimator used by PX4. It is better tested and supported than LPE, and should be used by preference.

The messages should be streamed at between 30Hz (if containing covariances) and 50 Hz. If the message rate is too low, EKF2 will not fuse the external vision messages.

The following MAVLink "vision" messages are not currently supported by PX4:

[GLOBAL_VISION_POSITION_ESTIMATE](#), [VISION_SPEED_ESTIMATE](#),
[VICON_POSITION_ESTIMATE](#)

Reference Frames

PX4 uses FRD (X **F**orward, Y **R**ight and Z **D**own) for the local body frame as well for the reference frame. When using the heading of the magnetometer, the PX4 reference frame x axis will be aligned with north, so therefore it is called NED (X **N**orth, Y **E**ast, Z **D**own). The heading of the reference frame of the PX4 estimator and the one of the external pose estimate will not match in most cases. Therefore the reference frame of the external pose estimate is named differently, it is called [MAV_FRAME_LOCAL_FRD](#).

Depending on the source of your reference frame, you will need to apply a custom transformation to the pose estimate before sending the MAVLink Vision/MoCap message. This is necessary to change the orientation of the parent and child frame of the pose estimate, such that it fits the PX4 convention. Have a look at the MAVROS [odom plugin](#) for the necessary transformations.

TIP

ROS users can find more detailed instructions below in [Reference Frames and ROS](#).

For example, if using the Optitrack framework the local frame has x and z on the horizontal plane (x front and z right) while y axis is vertical and pointing up. A simple trick is swapping axis in order to obtained NED convention.

If $x_{\{mav\}}$, $y_{\{mav\}}$ and $z_{\{mav\}}$ are the coordinates that are sent through MAVLink as position feedback, then we obtain:

$$\begin{aligned}x_{\{mav\}} &= x_{\{mocap\}} \\y_{\{mav\}} &= z_{\{mocap\}} \\z_{\{mav\}} &= -y_{\{mocap\}}\end{aligned}$$

Regarding the orientation, keep the scalar part w of the quaternion the same and swap the vector part x , y and z in the same way. You can apply this trick with every system - if you need to obtain a NED frame, look at your MoCap output and swap axis accordingly.

EKF2 Tuning/Configuration

Note: this is a quick overview. For more detailed information, check the [Using PX4's Navigation Filter \(EKF2\)](#)

The following parameters must be set to use external position information with EKF2 (these can be set in *QGroundControl* > **Vehicle Setup** > **Parameters** > **EKF2**).

Parameter	Setting for External Position Estimation
EKF2_EV_CTRL	Set <i>horizontal position fusion</i> , <i>vertical vision fusion</i> , <i>velocity fusion</i> , and <i>yaw fusion</i> , according to your desired fusion model.

Parameter	Setting for External Position Estimation
EKF2_HGT_REF	Set to <i>Vision</i> to use the vision as the reference source for altitude estimation.
EKF2_EV_DELAY	Set to the difference between the timestamp of the measurement and the "actual" capture time. For more information see below .
EKF2_EV_POS_X , EKF2_EV_POS_Y , EKF2_EV_POS_Z	Set the position of the vision sensor (or MoCap markers) with respect to the robot's body frame.

You can also disable GNSS, baro and range finder fusion using [EKF2_GPS_CTRL](#), [EKF2_BARO_CTRL](#) and [EKF2_RNG_CTRL](#), respectively.

TIP

Reboot the flight controller in order for parameter changes to take effect.

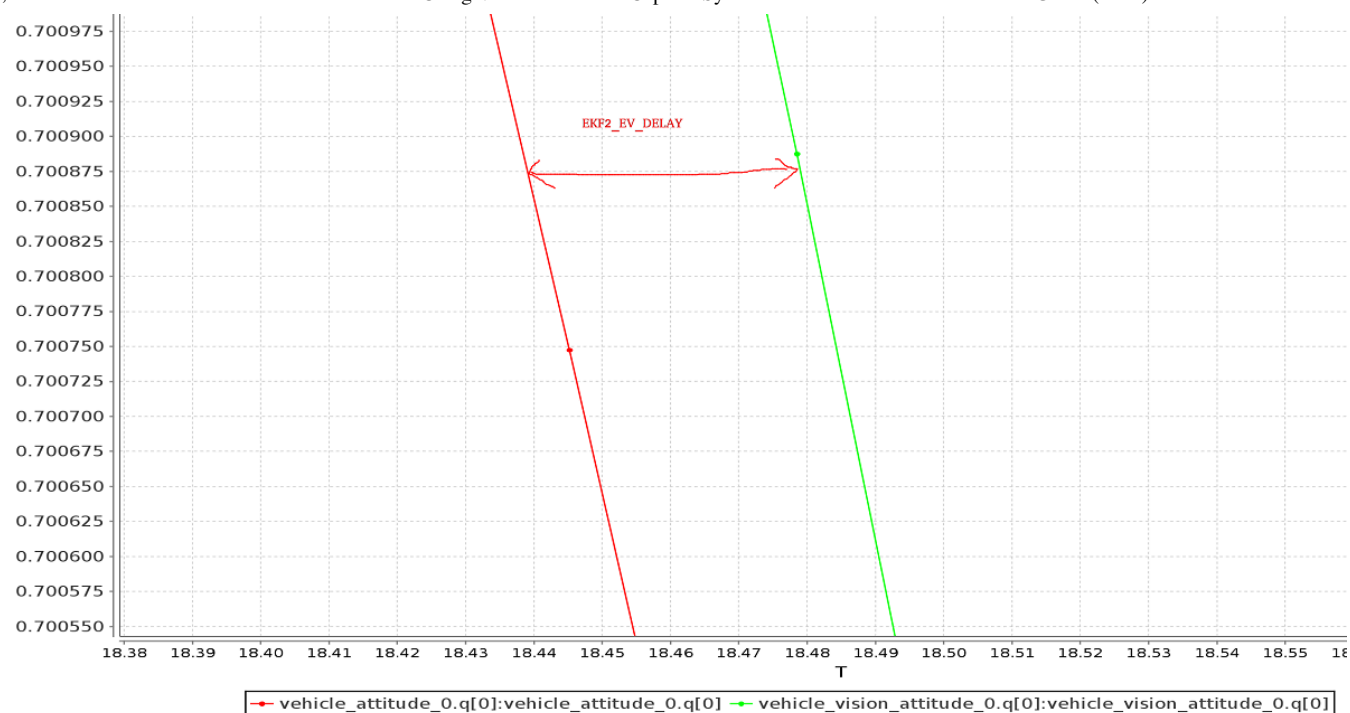
Tuning EKF2_EV_DELAY

[EKF2_EV_DELAY](#) is the *Vision Position Estimator delay relative to IMU measurements*.

Or in other words, it is the difference between the vision system timestamp and the "actual" capture time that would have been recorded by the IMU clock (the "base clock" for EKF2).

Technically this can be set to 0 if there is correct timestamping (not just arrival time) and timesync (e.g NTP) between MoCap and (for example) ROS computers. In reality, this needs some empirical tuning since delays in the entire MoCap→PX4 chain are very setup-specific. It is rare that a system is setup with an entirely synchronised chain!

A rough estimate of the delay can be obtained from logs by checking the offset between IMU rates and the EV rates. To enable logging of EV rates set bit 7 (Computer Vision and Avoidance) of [SDLOG_PROFILE](#).



INFO

A plot of external data vs. onboard estimate (as above) can be generated using [FlightPlot](#) or similar flight analysis tools. At time of writing (July 2021) neither [Flight Review](#) nor [MAVGCL](#) support this functionality.

The value can further be tuned by varying the parameter to find the value that yields the lowest EKF innovations during dynamic maneuvers.

LPE Tuning/Configuration

You will first need to [switch to the LPE estimator](#) by setting the following parameters: [LPE_EN](#) (1), [EKF2_EN](#) (0), [ATT_EN](#) (0).

INFO

If targeting [px4_fmu-v2](#) hardware you will also need to use a firmware version that includes the LPE module (firmware for other FMU-series hardware includes both LPE and EKF). The LPE version can be found in the zip file for each PX4 release or it can be built from source using the build command `make px4_fmu-v2_lpe`. See [Building the Code](#) for more details.

Enabling External Pose Input

The following parameters must be set to use external position information with LPE (these can be set in *QGroundControl* > **Vehicle Setup** > **Parameters** > **Local Position Estimator**).

Parameter	Setting for External Position Estimation
LPE_FUSION	Vision integration is enabled if <i>fuse vision position</i> is checked (it is enabled by default).
ATT_EXT_HDG_M	Set to 1 or 2 to enable external heading integration. Setting it to 1 will cause vision to be used, while 2 enables MoCap heading use.

Disabling Barometer Fusion

If a highly accurate altitude is already available from VIO or MoCap information, it may be useful to disable the baro correction in LPE to reduce drift on the Z axis.

This can be done by in *QGroundControl* by unchecking the *fuse baro* option in the [LPE_FUSION](#) parameter.

Tuning Noise Parameters

If your vision or MoCap data is highly accurate, and you just want the estimator to track it tightly, you should reduce the standard deviation parameters: [LPE_VIS_XY](#) and [LPE_VIS_Z](#) (for VIO) or [LPE_VIC_P](#) (for MoCap). Reducing them will cause the estimator to trust the incoming pose estimate more. You may need to set them lower than the allowed minimum and force-save.

TIP

If performance is still poor, try increasing the [LPE_PN_V](#) parameter. This will cause the estimator to trust measurements more during velocity estimation.

Enabling Auto Modes with a Local Position

All PX4 automatic flight modes (such as [Mission](#), [Return](#), [Land](#), [Hold](#), [Orbit](#))) require a *global* position estimate, which would normally come from a GPS/GNSS system.

Systems that only have a *local* position estimate (from MOCAP, VIO, or similar) can use the [SET_GPS_GLOBAL_ORIGIN](#) MAVLink message to set the origin of the EKF to a particular global location. EKF will then provide a global position estimate based on origin and local frame position.

This can then be used when planning and executing indoor missions, or to set a local return point, and so on.

Working with ROS

ROS is not *required* for supplying external pose information, but is highly recommended as it already comes with good integrations with VIO and MoCap systems. PX4 must already have been set up as above.

Getting Pose Data Into ROS

VIO and MoCap systems have different ways of obtaining pose data, and have their own setup and topics.

The setup for specific systems is covered [below](#). For other systems consult the vendor setup documentation.

Relaying Pose Data to PX4

MAVROS has plugins to relay a visual estimation from a VIO or MoCap system using the following pipelines:

ROS	MAVLink	uORB
/mavros/vision_pose/pose	VISION_POSITION_ESTIMATE	vehicle_visual_odometry
/mavros/odometry/out (frame_id = odom , child_frame_id = base_link)	ODOMETRY (frame_id = MAV_FRAME_LOCAL_FRD)	vehicle_visual_odometry
/mavros/mocap/pose	ATT_POS_MOCAP	vehicle_mocap_odometry

ROS	MAVLink	uORB
<pre>/mavros/odometry/out (frame_id = odom , child_frame_id = base_link)</pre>	<pre><u>ODOMETRY</u> (frame_id = <u>MAV_FRAME_LOCAL_FRD</u>)</pre>	<pre>vehicle_mocap_odometry</pre>

You can use any of the above pipelines with LPE.

If you're working with EKF2, only the "vision" pipelines are supported. To use MoCap data with EKF2 you will have to [remap](#) the pose topic that you get from MoCap:

- MoCap ROS topics of type `geometry_msgs/PoseStamped` or `geometry_msgs/PoseWithCovarianceStamped` must be remapped to `/mavros/vision_pose/pose`. The `geometry_msgs/PoseStamped` topic is most common as MoCap doesn't usually have associated covariances to the data.
- If you get data through a `nav_msgs/Odometry` ROS message then you will need to remap it to `/mavros/odometry/out`, making sure to update the `frame_id` and `child_frame_id` accordingly.
- The odometry frames `frame_id = odom`, `child_frame_id = base_link` can be changed by updating the file in `mavros/launch/px4_config.yaml`. However, the current version of mavros (`1.3.0`) needs to be able to use the tf tree to find a transform from `frame_id` to the hardcoded frame `odom_ned`. The same applies to the `child_frame_id`, which needs to be connected in the tf tree to the hardcoded frame `base_link_frd`. If you are using mavros `1.2.0` and you didn't update the file `mavros/launch/px4_config.yaml`, then you can safely use the odometry frames `frame_id = odom`, `child_frame_id = base_link` without much worry.
- Note that if you are sending odometry data to px4 using `child_frame_id = base_link`, then you need to make sure that the `twist` portion of the `nav_msgs/Odometry` message is **expressed in body frame, not in inertial frame!!!!**.

Reference Frames and ROS

The local/world and world frames used by ROS and PX4 are different.

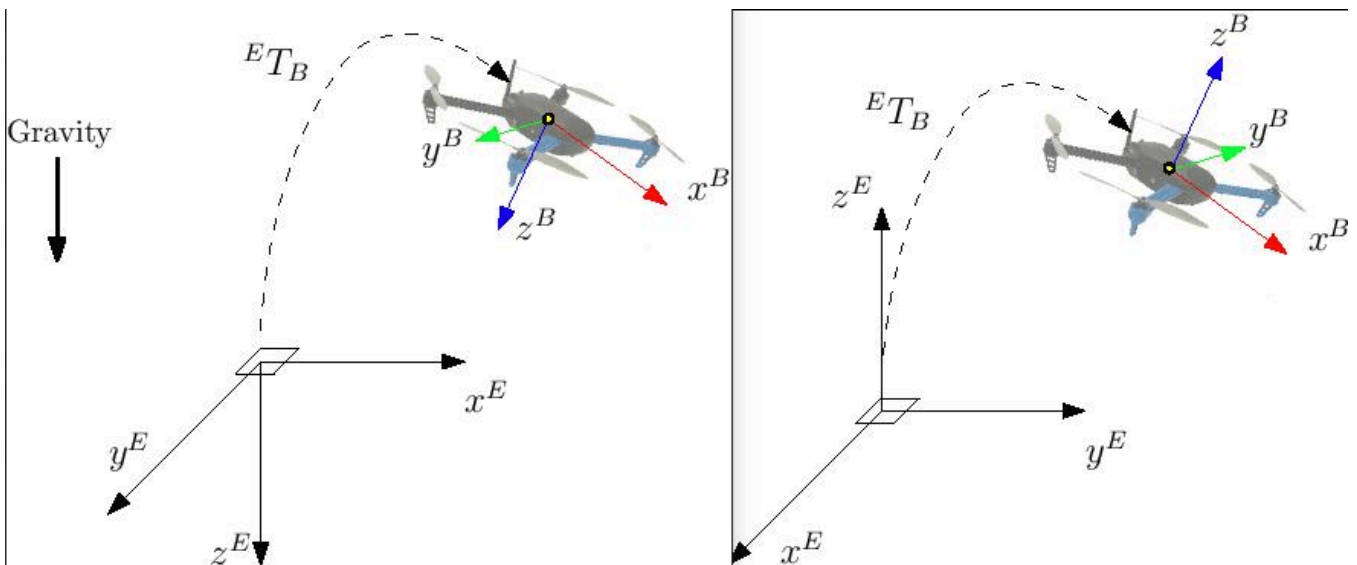
Frame	PX4	ROS
Body	FRD (X Forward, Y Right, Z Down)	FLU (X Forward, Y Left, Z Up), usually named <code>base_link</code>

Frame	PX4	ROS
World	FRD or NED (X North, Y East, Z Down)	FLU or ENU (X East, Y North, Z Up), with the naming being <code>odom</code> or <code>map</code>

TIP

See [REP105: Coordinate Frames for Mobile Platforms](#) for more information about ROS frames.

Both frames are shown in the image below (FRD on the left/FLU on the right).



With EKF2 when using external heading estimation, magnetic north can either be ignored and or the heading offset to magnetic north can be calculated and compensated. Depending on your choice the yaw angle is given with respect to either magnetic north or local x.

INFO

When creating the rigid body in the MoCap software, remember to first align the robot's local x axis with the world x axis otherwise the yaw estimate will have an offset. This can stop the external pose estimate fusion from working properly. Yaw angle should be zero when body and reference frame align.

Using MAVROS, this operation is straightforward. ROS uses ENU frames as convention, therefore position feedback must be provided in ENU. If you have an Optitrack system you can use [mocap_optitrack](#) node which streams the object pose on a ROS topic already in ENU. With a remapping you can directly publish it on `mocap_pose_estimate` as it is without any transformation and MAVROS will take care of NED conversions.

The MAVROS odometry plugin makes it easy to handle the coordinate frames. It uses ROS's tf package. Your external pose system might have a completely different frame convention that does not match the one of PX4. The body frame of the external pose estimate can depend on how you set the body frame in the MOCAP software or on how you mount the VIO sensor on the drone. The MAVROS odometry plugin needs to know how the external pose's child frame is oriented with respect to either the airframe's FRD or FLU body frame known by MAVROS. You therefore have to add the external pose's body frame to the tf tree. This can be done by including an adapted version of the following line into your ROS launch file.

```
<node pkg="tf" type="static_transform_publisher" name="tf_baseLink_external"
      args="0 0 0 <yaw> <pitch> <roll> base_link <external_pose_child_frame
```

Make sure that you change the values of yaw, pitch and roll such that it properly attaches the external pose's body frame to the `base_link` or `base_link_frd`. Have a look at the [tf package](#) for further help on how to specify the transformation between the frames. You can use rviz to check if you attached the frame right. The name of the `external_pose_child_frame` has to match the `child_frame_id` of your `nav_msgs/Odometry` message. The same also applies for the reference frame of the external pose. You have to attach the reference frame of the external pose as child to either the `odom` or `odom_frd` frame. Adapt therefore the following code line accordingly.

```
<node pkg="tf" type="static_transform_publisher" name="tf_odom_externalPose"
      args="0 0 0 <yaw> <pitch> <roll> odom <external_pose_parent_frame> 10
```

If the reference frame has the z axis pointing upwards you can attached it without any rotation (yaw=0, pitch=0, roll=0) to the `odom` frame. The name of `external_pose_parent_frame` has to match the `frame_id` of the odometry message.

INFO

When using the MAVROS *odom* plugin, it is important that no other node is publishing a transform between the external pose's reference and child frame. This might break the tf tree.

Specific System Setups

OptiTrack MoCap

The following steps explain how to feed position estimates from an [OptiTrack](#) system to PX4. It is assumed that the MoCap system is calibrated. See [this video](#) for a tutorial on the calibration process.

Steps on the *Motive* MoCap software

- Align your robot's forward direction with the [system +x-axis](#)
- [Define a rigid body in the Motive software](#). Give the robot a name that does not contain spaces, e.g. `robot1` instead of `Rigidbody 1`
- [Enable Frame Broadcast and VRPN streaming](#)
- Set the Up axis to be the Z axis (the default is Y)

Getting pose data into ROS

- Install the `vrpn_client_ros` package
- You can get each rigid body pose on an individual topic by running

```
roslaunch vrpn_client_ros sample.launch server:=<mocap machine ip>
```

sh

If you named the rigidbody as `robot1`, you will get a topic like

`/vrpn_client_node/robot1/pose`

Relaying/remapping Pose Data

MAVROS provides a plugin to relay pose data published on `/mavros/vision_pose/pose` to PX4. Assuming that MAVROS is running, you just need to **remap** the pose topic that you get from MoCap `/vrpn_client_node/<rigid_body_name>/pose` directly to `/mavros/vision_pose/pose`. Note that there is also a `mocap` topic that MAVROS provides to feed `ATT_POS_MOCAP` to PX4, but it is not applicable for EKF2. However, it is applicable with LPE.

INFO

Remapping pose topics is covered above [Relaying pose data to PX4](#)

(`/vrpn_client_node/<rigid_body_name>/pose` is of type `geometry_msgs/PoseStamped`).

Assuming that you have configured EKF2 parameters as described above, PX4 now is set and fusing MoCap data.

You are now set to proceed to the first flight.

First Flight

After setting up one of the (specific) systems described above you should now be ready to test. The instructions below show how to do so for MoCap and VIO systems

Check external estimate

Be sure to perform the following checks before your first flight:

- Set the PX4 parameter [MAV_ODOM_LP](#) to 1. PX4 will then stream back the received external pose as MAVLink [ODOMETRY](#) messages.
- You can check these MAVLink messages with the *QGroundControl* [MAVLink Inspector](#). In order to do this, yaw the vehicle until the quaternion of the [ODOMETRY](#) message is very close to a unit quaternion. ($w=1, x=y=z=0$)
- At this point the body frame is aligned with the reference frame of the external pose system. If you do not manage to get a quaternion close to the unit quaternion without rolling or pitching your vehicle, your frame probably still have a pitch or roll offset. Do not proceed if this is the case and check your coordinate frames again.
- Once aligned you can pick the vehicle up from the ground and you should see the position's z coordinate decrease. Moving the vehicle in forward direction, should increase the position's x coordinate. While moving the vehicle to the right should increase the y coordinate. In the case you send also linear velocities from the external pose system, you should also check the linear velocities. Check that the linear velocities are expressed in the *FRD* body frame reference frame.
- Set the PX4 parameter [MAV_ODOM_LP](#) back to 0. PX4 will stop streaming this message back.

If those steps are consistent, you can try your first flight.

Put the robot on the ground and start streaming MoCap feedback. Lower your left (throttle) stick and arm the motors.

At this point, with the left stick at the lowest position, switch to position control. You should have a green light. The green light tells you that position feedback is available and

position control is now activated.

Put your left stick at the middle, this is the dead zone. With this stick value, the robot maintains its altitude; raising the stick will increase the reference altitude while lowering the value will decrease it. Same for right stick on x and y.

Increase the value of the left stick and the robot will take off, put it back to the middle right after. Check if it is able to keep its position.

If it works, you may want to set up an [offboard](#) experiment by sending position-setpoint from a remote ground station.

[Edit on GitHub](#)

Previous page

[ROS/MAVROS Installation on RPi](#)

Next page

[Community](#)